ANTEATER CHESS
SOFTWARE SPECIFICATIONS
VERSION 1.0

TEAM 21
Naveen Khan
Mia Ness
Austin Nguyen
Queency Tan
Aman Macwan
Gurvin Dhanjal

TABLE OF CONTENTS

1. SOFTWARE ARCHITECTURE OVERVIEW

1.1 Main Data Types and Structures..... 2
1.2 Major Software Components..... 3
1.3 Module Interfaces..... 4
1.4 Overall Program Control Flow..... 4

2. INSTALLATION

2.1 System Requirements and Compatibility.....6
2.2 Setup and Configuration.....6
2.3 Building, Compilation, and Installation.....6

3. DOCUMENTATION OF PACKAGES, MODULES, AND INTERFACES

3.1 Detailed Description of Data Structures.....7
3.2 Detailed Description of Functions and Parameters..... 10
3.3 Detailed Description of Input and Output Formats.....12

4. DEVELOPMENT PLAN AND TIMELINE

4.1 Partitioning of Tasks.....13
4.2 Team Member Responsibilities.....15

5. Back Matter

5. Copyright..... 16
6. Reference..... 17
7. Reference..... 18

Software Architecture Overview

1.1 MAIN DATA TYPES AND STRUCTURES

The system is built around a set of data structures that model the complete state of a chess game. These structures provide a consistent and easy to follow representation of the game while being able to support advanced rule enforcements.

The core of the system is denoted by two enumerated types, Color and Rank. The Color type identifies the player ownership, while the Rank type defines the category of each piece which includes both the standard chess pieces and custom pieces (Anteater). These representations allow the engine to enforce the rules dynamically based on the piece type and player rather than integer values.

Each piece in the game is stored using a Piece struct, which keeps track of what type of piece it is and what color it belongs to. It also allows each piece to follow its own movement and capture rules while still being handled in the same general way by the program.

The board positions are represented using a Pos structure, which maps the readable chess coordinates into internal values that the engine can read. Moves are encapsulated using the Move structure, which supports both internal processing and notation output.

The board itself is stored as a two dimensional array. Each spot on the board either points to a piece or is empty. This makes it easy to check, update, and display the board during the game.

The program also uses a stack to keep track of moves. Every move made during the game is stored in this stack. This can be used for features like undoing moves or keeping a history of the game.

1.2 Major Software Components

This system follows a module blueprint in which each component is responsible for a certain aspect of the chess game. This design is done to reduce coupling and to keep it easily maintainable if any necessary updates or replacements are needed without affecting the overall engine.

The major components include:

- Core Engine (main.c)
    - Central controller for the whole program, coordinating all the modules and managing the game cycle
- Board Management (board.c)
    - Handles the board initialization, state updates and rendering of the game board.
- Piece Logic (piece.c)
    - Defines piece construction and provides piece specifications through virtual function tables
- Move Processing (move.c)
    - Responsible for executing the moves, updating the game and triggering actions like captures.
- Rules Engine (rules.c)
    - Implements all the game rules like move validation, check detection, checkmate evaluation and special mechanic moves like Anteater capture.
- Input Handler (input.c)
    - Parses and validates user input, converting external commands into internal representations.
- Logging System (log.c)
    - Records all of the game actions in a structured format to traceability and replay.
- History Manager (undo.c)
    - Maintains a stack of moves to support undo functionality and state rollback.

The module hierarchy of the system is shown below:

Core Engine (main.c)

- Input Handler (input.c)
- Board Management (board.c)
- Move Processing (move.c)
    - Piece Logic (piece.c)
- Rules Engine (rules.c)
    - Board Management (board.c)
    - Piece Logic (piece.c)
- Logging System (log.c)
- History Manager (undo.c)

1.3 Module Interfaces

Core Engine (main.c)

```
int main();
```

Board Management (board.c)

```
void initBoard();
void displayBoard();
Piece* getPiece(Pos p);
void setPiece(Pos p, Piece* piece);
```

Piece Logic (piece.c)

```
void pieceCtor(Piece* mPiece, Color color, Rank rank, const Piece_Vtable* vtable);
void moveCtor(Move* mMove, Rank rank, Pos newPos);
void posCtor(Pos* mPos, char file, int rank);
```

Move Processing (move.c)

```
void move(int startRow, int startCol, int endRow, int endCol);
```

Rules Engine (rules.c)

```
void capture(int row, int col);
bool anteaterCapture(int row, int col);
void addPoints(const Piece* captured);
void castle(Pos rookPos, Color color);
bool check(Color color, int row, int col);
void promotion(int row, int col, char pieceType);
```

Input Handler (input.c)

```
Move parseInput(const char* input);
bool isValid(const char* input);
```

Logging System (log.c)

```
void logMove(FILE* logFile, Move m);
```

History Manager (undo.c)

```
void pushMove(Move m);
Move popMove();
```

1.4 Overall Program Flow

1. Program start: main() initializes all modules
2. New Game screen is displayed:

- Player selects color (White or Black)
- Player selects difficulty (Easy, Medium, or Hard)
- Player clicks "Start Game"

3. Board is initialized (initBoard()) and displayed(displayBoard())
4. Timer starts for White's turn
5. Game loop begins, repeating until checkmate, stalemate, or quit:
    a. If it is the human player's turn:
        i. Player clicks a piece, legal moves are highlighted on the board
        ii. Player clicks a destination square
        iii. Validate move legality (rules.c - validateMove())
        iv. If invalid, display error message and re-prompt
        v. If valid, execute the move (move.c - move())
            - If a piece is captured, call capture() or anteaterCapture()
            - If castling, call castle()
            - If en passant, call enPassant()
            - If promotion, call promotion() then promote()
    b. If it is the CPU's turn:
        i. AI selects a legal move (ai.c)
        ii. Execute the move (move.c - move())
    c. Update point totals if capture occurred (rules.c - addPoints())
    d. Push move onto history stack (undo.c - pushMove())
    e. Log the move to action text file (log.c - logMove())
    f. Check for check, checkmate, or stalemate (rules.c - check())
    g. Display updated board (board.c - displayBoard())
    h. Stop timer and switch turns
    i. If undo requested, call popMove() and reverse board state
6. Game end - displays winner or stalemate
7. Player chooses:
    - New game: return to step 2
    - Quit: program exit

2.1. System requirements and compatibility

Before installing and running the program please make sure your system has and meets the following requirements:

- Operating System (Linux)
- Compiler (gcc)
- Disk Space
- UCI EECS lab machines

2.2 Setup and configuration

In order to install the program, please follow the steps below:

Step 1: Extract Files

a. To extract the files, type the following into your terminal:
"gtar xvzf Chess_V1.0_src.tar.gz"

Step 2: Change the Directory:

a. After you have finished extracting the files, change into the program directory. To change into the program directory, type the following into your terminal:
"cd chess"

Step 3: Compile the program:

a. Once you have changed to the correct directory, you can now compile the program. To compile the program, type the following into your terminal:
"make"

Step 4: Run the Program:

- The last and final step is to run the program. To run the program, type the following into your terminal:
"./chess"

2.3. Building, compilation, and installation

After the project has been extracted and the developer has navigated to the chess directory. The software, Makefile, will automate the compilation and link of all source files into a single executable. The make command will compile the program, generate the necessary object files, and produce the final executable in the output directory. To remove compiled files and reset the project directory, the make clean command can be used. No separate installation process is needed after the files are compiled. Once the build process is complete, the executable can be run directly from the project directory.

3.1 Description of Data Structures

ENUMERATED TYPES

Two custom enumerations are defined to represent piece color and piece type.

```
typedef enum {
      BLACK = 0;
      WHITE = 1;
} Color;
```

```
typedef enum {
      KING = 0;
      QUEEN = 1;
      ANTEATER = 2;
      BISHOP = 3;
      KNIGHT = 4;
      ROOK = 5;
      ANT = 6;
} Rank;
```

VIRTUAL TABLES

In this project we will also be using a virtual table structure named “Piece_Vtable’ in order to support overriding and piece-specific behavior. This structure will store function pointers for all the actions a piece can perform. As of now, those actions are “move” and “capture”

```
typedef struct {
      void (*move)(int startRow, int startCol, int endRow, int endCol);
      void (*capture)(int row, int col);
} Piece_Vtable;
```

PIECE STRUCTURE

Each of our chess pieces will be represented by a struct named Piece. The piece structure will have a field of type ‘Rank’ named rank to represent the type of the piece (king, queen, ant, etc.). It will also have another field of type ‘Color’ to signify whether the piece is white or black. Finally, it will have a pointer to an object of type ‘Piece_Vtable’ in order to support function overriding.

```
typedef struct {
    Rank rank; //KING QUEEN ANTEATER BISHOP KNIGHT ROOK ANT
    Color color; //WHITE BLACK
    int pointVal;
    Piece_Vtable *vtable;
} Piece;
```

POSITION STRUCTURE

A structure named ‘Pos’ will be used to represent board coordinates. The ‘file’ field will be stored as a character and represent the column of the board and the ‘rank’ field will be stored as an integer and represent the row of the board.

```
typedef struct {
    char file;
    int rank;
} Pos;
```

MOVE STRUCTURE

Our moves will be represented by a struct named ‘Move’ which will contain a c-string representing the move. The moves will be stored following standard chess algebraic notation with slight modifications to account for special pieces like anteaters and ants.

```
typedef struct {
        char move[4];
} Move;
```

```
static const char pieceNotation[7] = {'K', 'Q', 'A', 'B', 'N', 'R', 'T'};
//A is for anteater and T is for ant. Everything else follows standard
chess algebraic notation
```

To simplify the conversion from a piece's rank to a piece's notation, a constant char array will be made where the index will correspond to the piece's rank.

```
void moveCtor(Move* mMove, Rank rank, Pos newPos){
    mMove->move[0] = pieceNotation[rank]; //gets the notation for the Piece
    mMove->move[1] = nextPos.file;
    mMove->move[2] = nextPos.rank + '0';
    mMove->move[3] = '\0';
}
```

The current implementation only accounts for simple moves to empty squares. The future implementation will be expanded to support longer notations for special moves such as capturing, checking, castling, promoting, and etc.

BOARD REPRESENTATION

The game board is represented as a two-dimensional array of pointers to Piece objects. Each element is associated with a specific square on the board with the first index representing the rank and the second index representing the file of the square. If that square contains a piece, the element will point to a Piece object. If it does not contain a piece, it will point to NULL.

```
Piece* board[8][10];
```

The board is defined as 8 x 10 instead of the standard 8 x 8 following the custom anteater chess project specification which uses a non-symmetrical board layout.

LOG OF MOVES : STACK

The move history will be stored using a stack structure with a data type of 'Move', This will allow us to implement "undo's" as popping from the stack.

The structures and additional features such as a point counter to track captured pieces will be added later on.

3.2 Detailed Description of Functions and Parameters

Function 1: Move
void move(int startRow, int startCol, int endRow, int endCol);

- Verifies that the selected piece belongs to the current player, checks whether the move is legal based on the piece type, and ensures the move does not result in check.
- **Output**: moves the chosen piece to the chosen location, calls the capture function if necessary, adds the move to the action text file

Function 2: Capture
void capture(int row, int col);

- Captures a piece and removes the captured piece from the board
- **Parameters**: position of the captured piece
- **Outputs**: removes a captured piece, calls point counter, and adds name of the piece to a text file, adds the capture to the action text file

Function 3: Anteater Capture
bool anteaterCapture(int row, int col);

- Captures pieces and removes the captured ant piece(s) from the board
- **Parameters:** current position of the anteater
- **Inputs**: takes move function of anteater, piece color
- **Outputs**: removes a captured ant piece, calls point counter, adds name of the piece to a text file, returns a boolean value if it can capture again, adds the capture to the action text file

Function 4: Point Counter
void addPoints(const Piece* captured)

- Adds point depending on captured piece
- **Inputs**: the captured piece
- **Outputs**: adds the points of captured piece to the total value in the points

Function 5: Castle
void castle(Pos rookPos, Color color)

- Switches position of the king and a rook
- **Inputs**: position of the rook, piece color
- **Outputs**: moves the king two spaces closer to the rook and puts the rook one space on the other side of the king, adds the move to the action text file
- **Logic**: checks if the spaces between rook and king are empty, checks if the king will be in check if castle is complete, checks the direction the king is moving, finds the new location of the king and rook
- **Error**: If move isn't legal or the king will be in check, the piece will not move, (Text message will be prompted…TBD)

Function 6: Check
bool check(char color, int row, int col);

- Checks for check, checkmate, or stalemate
- **Parameters**: takes in color and king position to see if the king can move or not
- **Outputs**: displays which color is in check, returns boolean of true or false for checkmate or stalemate

Function 7: Promotion
void promotion(int row, int col, char pieceType);

- **Parameter:** row, col: position of the pawn, pieceType: promote to ('Q', 'R', 'B', 'N', 'A')
- **Outputs**:changes the pow

Function 8: Piece Constructor
void pieceCtor(Piece* mPiece, Color color, Rank rank, const Piece_Vtable* vtable)

- A constructor to make a Piece object
- **Parameters**: takes in a pointer to the piece object we want to instantiate as well as a color, rank, and virtual table containing the piece's specific behaviour
- **Outputs**: initializes the Piece argument

Function 9: Move Constructor
void moveCtor(Move* mMove, Rank rank, Pos newPos)

- A constructor to make a Move object
- **Parameters**: takes in a pointer to the Move object we want to instantiate as well as a rank, and Pos object containing the piece's new position.
- **Outputs**: initializes the passed in Move object.

Function 10: Position Constructor
void posCtor(Pos* mPos, char file, int rank)

- A constructor to make a Pos object
- **Parameters**: takes in a pointer to the Pos object we want to instantiate as well as a char named file containing the piece's column, and an integer named rank containing the piece's row number.
- **Outputs**: initializes the passed in Pos object.

3.3 Detailed Description of Input and Output Formats

Input
Moves are entered using coordinate notation representing the starting and ending positions of a piece.

{F2, F4}

The first coordinate represents the starting position. The second coordinate represents the destination

This format is used for both user input and internal processing.

Output
All moves are recorded in a text file using the same coordinate format.
{
F2 F4
G8 F6
E7 E5
F4 E5 (capture)
}

After each move, the board is updated and displayed, the move is logged, and the program indicates if a player is in check, checkmate, or stalemate.

4.1 Partitioning of Tasks

TASK 1: CORE ENGINE AND MAIN PROGRAM (main.c)

The central controller that coordinates all modules, manages the game loop, and handles turn alternation between the human player and the computer opponent. This module initializes the board, invokes the input handler, dispatches moves to the rules engine, and triggers the display after each turn.

TASK 2: BOARD MANAGEMENT (board.c)

Handles initialization of the 8×10 board array with the correct starting layout, state updates after each move, and rendering of the board to the terminal in ASCII format. This module also provides utility functions for querying and modifying individual squares.

TASK 3: PIECE LOGIC (piece.c)

Defines the Piece struct, Color and Rank enumerations, and the virtual function table (Piece_Vtable) that enables piece-specific movement and capture behavior. This module provides constructors for each piece type and encapsulates the rules governing how each piece can move on the board.

TASK 4: MOVE PROCESSING (move.c)

Responsible for executing validated moves on the board, updating piece positions, handling standard captures, and recording each move using the Move struct and coordinate notation. This module bridges the gap between the Rules Engine (which validates legality) and Board Management (which updates state).

TASK 5: RULES ENGINE (rules.c)

Implements all game rule enforcement including move validation, legal move generation, check detection, checkmate and stalemate evaluation, and all special moves: castling, en passant, pawn promotion, and the Anteater chain capture mechanic. The Anteater's chain capture logic — where the Anteater eats consecutive adjacent enemy pawns along cardinal directions in a single move — is the most complex rule and requires careful implementation and testing.

TASK 6: INPUT HANDLER (input.c)

Parses and validates user input entered in coordinate notation (e.g., F2 F4), converting external commands into internal Pos and Move representations. Also handles special input commands such as castling notation (O-O, O-O-O), undo requests, and quit commands. Provides clear error messages for malformed or invalid input.

TASK 7: LOGGING SYSTEM (log.c)

Records all game actions in a human-readable text file using coordinate notation. Each move, capture, and special action is appended to the log in sequential order. This satisfies the project requirement for a persistent move log and supports debugging and game replay.

TASK 8: HISTORY MANAGER / UNDO (undo.c)

Maintains a stack-based history of all moves to support the undo feature. Each entry must store enough state to fully reverse a move, including restoring captured pieces and reverting special moves such as castling or Anteater chain captures. Undo for chain captures requires restoring all pawns consumed in a single Anteater move.

TASK 9: AI / COMPUTER PLAYER (ai.c)

Implements the automatic computer opponent that selects moves on behalf of the non-human player. The AI module receives the current board state and generates a legal move within the required time limit (less than 1 minute per move). Initial implementation will use a basic evaluation function with look-ahead; further refinement may include minimax with alpha-beta pruning if time permits.

TASK 10: GRAPHICAL USER INTERFACE (gui.c)

Replaces the initial ASCII text-based display with a graphical board interface as specified in the project requirements. The GUI will render the 8×10 board with piece icons, highlight legal moves, and support mouse or keyboard input for move entry. Development begins after the text-based interface is stable, following the GUI programming techniques introduced in Lecture 4.

TASK 11: TESTING AND DEBUGGING

Development of automated test cases and scripts to validate correct rule enforcement across all piece types, special moves, and edge cases. Testing is especially critical for the Anteater chain capture, castling across the wider 10-column board, en passant, and promotion.

TASK 12: DOCUMENTATION

Preparation and maintenance of all written deliverables including the User Manual (Chess_UserManual.pdf), Software Architecture Specification (Chess_SoftwareSpec.pdf), README files, and inline source code comments. Documents will be updated regularly to reflect current progression.

4.2 Team Member Responsibilities

TEAM MEMBER	MAIN RESPONSIBILITIES	SHARED OR SECONDARY RESPONSIBILITIES
QUEENCY TAN	piece logic (piece.c), rules engine (rules.c), special moves detection	core engine (main.c), gui design and layout,coding quality control
MIA NESS	rules engine (rules.c), check/checkmate functionality + stalemate detection	core engine (main.c), document quality control, coding quality control
AUSTIN NGUYEN	ai/computer player (ai.c), move processing (move.c)	core engine (main.c), ai optimization, coding quality control
NAVEEN KHAN	input handler (input.c), documentation (title page, back matter)	core engine (main.c), integrating testing,coding quality control
AMAN MACWAN	board management (board.c), graphical user interface (gui.c)	core engine (main.c), behavioral testing for pieces, coding quality control
GURVIN DHANJAL	logging system (log.c),history management /undo (undo.c), script testing	core engine (main.c), makefile maintenance, git control, coding quality control

WHILE SPECIFIC MODULES HAVE BEEN ASSIGNED TO INDIVIDUAL MEMBERS, ALL MEMBERS WILL MAKE CONTRIBUTIONS TO THE CREATION OF THE FINAL PRODUCT.

5. COPYRIGHT

© 2026 Team 21 — EECS 22L, University of California, Irvine. All rights reserved.

This document and the accompanying Anteater Chess software are produced as coursework for EECS 22L, Spring 2026. No part of this document or its associated source code may be reproduced, distributed, or transmitted in any form without the prior written permission of the authors, except for academic evaluation by course instructors and teaching assistants.

Anteater Chess is an original adaptation of the traditional chess game, developed exclusively for educational purposes at the University of California, Irvine. The Anteater piece and associated rules are defined by the EECS 22L course specification authored by Prof. Rainer Doemer.

6. REFERENCES

Doemer, R. “EECS 22L: Project 1 — The Game of Anteater Chess.” University of California, Irvine, March 30, 2026.

Yuan, C. “EECS 22L: Chess Software Specification Grading Criteria.” University of California, Irvine, April 5, 2026

GCC, the GNU Compiler Collection. Free Software Foundation. https://gcc.gnu.org/

GNU Make Manual. Free Software Foundation. https://www.gnu.org/software/make/manual/

Git — Distributed Version Control System. https://git-scm.com/

7. INDEX

A

Action Text File, 5, 10
addPoints (Function 4), 4, 5, 10
Algebraic Notation, 8
Anteater Capture (Function 3), 10
Anteater Chess, 1, 7, 13, 14
Anteater Piece, 13

B

Bishop, 7, 8
Board Management (board.c), 3, 4, 13

C

Capture (Function 2), 10
Captured Piece Tracking, 10
Castle / Castling (Function 5), 4, 5, 10
Check (Function 8), 11
Checkmate, 5, 11, 12
Color (enum), 2, 7
Compile (make), 6
Coordinate Notation, 12
Core Engine (main.c), 3, 4, 13

E

En Passant (Function 6), 10
End Message (Function 7), 8
Extract Files (gtar), 6

F

File Notation (columns a–j), 5
Function Prototypes, 4, 10

G

GCC Compiler, 6
Get Position (Function 13), 8
Git (version control), 15
Graphical User Interface (gui.c), 14, 15

H

Highlighting, 11
Hint Button, 10
History Manager (undo.c), 3, 4, 14, 15

I

Input Handler (input.c), 3, 4, 13, 15
Installation, 6

K

King, 7, 8, 10
Knight, 7

L

Legal Move Enforcement, 4, 5, 10
Linux, 6
Logging System (log.c), 3, 4, 14, 15

M

Make Utility, 6, 7
Makefile, 6, 7, 15
Module Hierarchy, 3
Module Interfaces, 4
Move (Function 1), 10
Move Log, 12, 14
Move Processing (move.c), 3, 4, 13, 15
Move Structure (Move struct), 2, 8

N

New Game (Function 11), 10

O

Output Format, 12

P

Pair Programming, 12
Pawn / Ant, 7, 8, 10
Piece Constructor (Function 16), 11
Piece Logic (piece.c), 3, 4, 13, 15
Piece Structure (Piece struct), 2, 8
Piece_Vtable (virtual table), 7, 8
pieceNotation (char array), 8, 9
Point Counter (Function 4), 10
Pos Structure, 2, 8
Program Control Flow, 4, 5
Promotion (Function 15), 5, 11

Q

Queen, 7

R

Rank (enum), 2, 7
Rook, 7, 10
Rules Engine (rules.c), 3, 4, 13, 15
Run Program (./chess), 6

S

SetPosition (Function 14), 8
Software Architecture Overview, 2
Special Moves, 5, 10
Stack (move history), 2, 9
Stalemate, 5, 11, 12
System Requirements, 6

T

Team Member Responsibilities, 15
Testing and Debugging, 14
Timer/Change Turn (Function 9), 10
Tournament, 11
Turn (Function 10), 10

U

Undo (Function 12), 11, 12
Uninstalling, 3
Usage Scenario, 9

W

Winner Message, 5